

KushHR — Independent Audit Summary

Plain-language results of **three** independent AI security, quality & production-readiness audits of the KushHR application.

Phase-13 exit check · Audits run 2026-07-08 → 07-13 · Three systems, each run blind to the others: **Codex (GPT-5)**, **Fable 5 / Opus**, and **Sol (GPT-5.6)** · Full detail: *docs/checks/*-audit-*.md* · Action list: *docs/checks/audit-remediation-plan.md*

1 The bottom line

A strong result for an app built largely by AI — now confirmed by three independent reviewers who never saw each other's work. None found a data breach, a way to log in without a password, a leaked secret, or a way to break out and run code. The security foundations are solid: every data table is access-controlled, permissions are always read from the live database (never a stale token), the master key stays server-side, and files are private.

What they found is a **thin band of real "integrity" gaps** — mostly things a normal user clicking through the app can't reach, but a technical user sending hand-crafted requests could, plus one server setting switched the wrong way. The **three passes agreed on the important items** (strong confidence) and each pass **earned its keep**: the second and third both, independently, caught the three most serious issues the *first* pass missed — including the open sign-up door and two leave-balance bugs. The third reviewer (Sol) also did a **production-readiness review** (Section 4) that the first two didn't.

NO DATA BREACH

NO LOGIN BYPASS

NO LEAKED SECRETS

1 OPEN SIGN-UP DOOR

3 LEAVE-BALANCE BUGS

READINESS: 2.5 / 5 PILOT-GRADE

2 Must fix — do these first

Seven items with real-world consequences. All are fixable; several are small.

Anyone on the network can create their own account **2 OF 3**

A server setting (**self-registration**) is switched on. Someone who can reach the app — without any admin inviting them — can sign themselves up and land inside as a regular employee, seeing the staff directory, work emails, leave pages and more. It defeats the whole "admins create all accounts" rule. Two reviewers found this independently; the first pass missed it. **Fix:** [flip one setting off \(and the same default in the shared template\)](#), [plus a deploy check that fails if sign-up is still open](#). Minutes of work — the single highest-value fix in the audit.

Cancelling leave can return the wrong number of days **2 OF 3**

When approved leave is cancelled, the system **recalculates** the days instead of returning the exact amount it originally took. If a public holiday was added to the calendar in between, the employee is silently short-changed (or over-credited) — and even the audit log records the "right" number, so the drift is invisible. (Third reviewer added: multi-year leave and a "warning but save anyway" path make it worse.) **Fix:** [refund the exact amount that was deducted at approval](#); [never save a cancellation that can't refund cleanly](#). Add a test that proves it.

The "working days" setting doesn't actually do anything **1 OF 3**

Admins can configure which days count as the working week, and the screen confirms it saved — but **no part of the leave calculation ever reads it**. It's hard-coded to Mon–Fri everywhere. Any company that sets, say, a six-day week will have every leave balance quietly wrong. **Fix:** [either make the setting actually drive the calculation, or remove the control and state Mon–Fri is fixed](#). Don't leave a switch that lies.

Performance reviews: forged writes + early peeks ALL 3

All three landed on the same root cause: performance-review data has **no database-level guard** — the app code is the only thing checking permissions. So a manager sending a hand-crafted request could file a review into a closed/draft cycle, an employee could read their **draft** score before the manager submits it, and (third reviewer) a review's audit trail can name the wrong employee and a "frozen" cycle isn't actually frozen for progress edits. Normal clicking can't do this; crafted requests can. **Fix:** [add proper checks on the write and read paths, ideally backed by the database itself](#).

Approved leave can be recorded without deducting the balance 1 OF 3

The step that subtracts days from a balance only runs when leave is **changed** to approved, not when it's **created** as approved. An admin (only an admin) entering leave directly as "approved" skips the deduction entirely. **Fix:** [block direct "approved" entries, or route them through the same deduction step](#).

A manager could rewrite leave dates while approving 2 OF 3

The database lets a manager's approval request change **which dates** (and even which employee) a leave covers, not just approve/reject it. With a crafted request, a manager could turn a 2-day request into a 3-week one and drain the employee's balance. The app code prevents this today, but there's no database backstop like there is for other sensitive fields. **Fix:** [lock down which columns an approval is allowed to touch](#).

A failed employee edit can still change someone's role — untracked ALL 3

When an admin edits an employee, the **role change saves first**. If a later part of the save fails, the screen says "failed" but the person's role has already changed — with **no audit record** of it. You also can't tell from the log "who was promoted to admin, and when." **Fix:** [make the whole edit succeed-or-fail together, and log role changes as their own event](#).

3 Worth fixing — hardening & smaller correctness

Audit log fails quietly ALL 3

If writing an audit record fails, the action still goes ahead with no trail. The third reviewer rates this a release blocker and notes the fix must make the change and its record save *together*, not just retry the record. Important for payroll accountability.

Staff can bypass the file-upload rules SOL

The per-category upload rules (which file types are allowed where, plus the audit record) live only in the app. A technical user can upload a file straight to storage and register it directly, skipping those checks. [Route all uploads through the server path; remove the direct-write shortcut](#).

Uploads aren't scanned for malware SOL

Uploads are checked by their claimed type/extension, not their actual contents, and there's no virus scan. A malicious file named `policy.pdf` could be stored and later handed to an HR user. Add content/signature checks + scanning.

A "preview" tool can hog the server SOL

A leftover working-days preview accepts unlimited date ranges (e.g. year 1 to 9999), so one request can spin millions of loops and slow the app for everyone. It's also unused. [Delete it \(or bound the dates\)](#).

Logout doesn't confirm it worked SOL

If the sign-out call fails, the app still shows "logged out" while the session stays alive — a risk on shared computers. Check the result and clear the session.

Manager salary view isn't re-checked ALL 3

The report showing managers their team's salary summary doesn't re-confirm the person is still a manager — a recently-demoted one could keep seeing it. One-line database check.

Missing database guardrails on numbers ALL 3

Salary, currency, leave balances, file sizes are only validated in the app, not the database — a direct entry could store a negative salary or "DOGE" as a currency. Add database-level checks.

"Nothing happened" recorded as success 2 OF 3

Approve/reject and some delete actions log a success (and send emails) even when they changed no rows — e.g. two managers acting at once, or a stale button. Confirm a row actually changed before recording it.

Spreadsheet-formula risk in exports 2 OF 3

Exported reports don't neutralise a leading =/+ in names/notes, so a crafted value could run when an admin opens the file in Excel. Simple escaping fix.

Password changes aren't logged 1 OF 3

The request to reset a password is logged, but the actual change isn't — including for admin accounts. Add a record on completion.

Search boxes can break on punctuation ALL 3

Admin search builds its filter by pasting text in raw, so a name like "Doe, Jane" breaks it. Harmless today (admin-only) but fragile. Use safe query building.

Auto-RLS safety net can fail silently SOL

A safeguard that auto-locks new tables swallows its own errors — a future table could ship unprotected. Make it fail loudly, and add a check that verifies every table is locked.

Other smaller items MIXED

Row-existence "oracle" (uniform error messages), password-reset endpoint spam (proxy-only ingress), and un-refunded direct rejections — all on the list, lower urgency.

4 Is it ready for real staff data? — production-readiness review

The third reviewer (Sol) stepped back from bug-hunting and rated KushHR as professional software for handling real HR/payroll data:

2.5 / 5

Pilot-grade, not yet production-grade. More professional than a typical solo project — coherent layout, strong security docs, real security tests, a deploy runbook, careful human review. But the **operational safety net** doesn't yet match how sensitive HR/payroll data is. (Scale: 1 prototype · 2 MVP · **3 controlled production** · 4 mature · 5 continuously assured.)

The reviewer was explicit that this is a small internal tool (~15–20 users) that does *not* need big-company machinery — no microservices, Kubernetes, or an ops team. One app + one database + Docker is a legitimate professional setup. These are the gaps that actually matter at this size, before pointing real staff data at it:

Close the front door & the extra doors

Turn off self-registration (with a deploy check), remove demo/test accounts from the real setup, and expose the app only through the secure proxy (not its raw ports).

Fix the leave-balance & role bugs from Section 2

These are the core "the numbers must be right" and "permissions must be tracked" items.

Make changes and their audit record save together

So a salary/role/leave change can never happen without a matching trail; alert if the audit trail is unhealthy.

Backups you can actually restore from

Backups currently sit on the **same machine** as the live system (one disk failure loses both), and the restore script hides its own failures and can report success after a partial restore. Move backups off-host, make restore fail loudly, and do one full practice restore.

Make "green CI" mean something

Automated checks don't currently build the app or run the security tests — so a passing build doesn't prove the app works or that permissions hold. Add a real build + a throwaway-database test run, required before merging.

See failures before users report them

Errors only go to the console today. Add centralized error/log collection and a few alerts (errors, audit failures, sign-up anomalies, disk/backup health).

A short privacy/operations pack

Who owns the data, who to call in an incident, what's kept and for how long, an access review, and whether admins need two-factor login. Bank/tax/ID fields are currently stored as plain text — decide encryption deliberately.

These are separate from the code fixes and several (backups, CI, monitoring) can be done in parallel. The full ranked list with effort estimates is in the engineering action plan.

5 Housekeeping — quality, tidy-up & speed

No user-facing risk; these reduce future bugs and keep the codebase honest. All three audits flagged most of them.

- **Duplicated logic:** the working-days calculation, search resolvers, and several helpers exist in 3–5 divergent copies that have started to disagree. Consolidate to one each (this also unblocks the "working days" fix above).
- **Very large files:** a few files carry thousands of lines of unrelated logic, which is exactly why a fix landed in one place and was missed in its twin. Split by feature.
- **Notification emails block the user:** the app waits on sending email before finishing an action, with no timeout — a slow mail server hangs the page. Move email off the response and add a timeout.
- **Untyped database access:** hundreds of unchecked type assertions mean a renamed column compiles fine and breaks only at runtime. Generate the database types.
- **Dead code & unused packages** (~340+ lines, several dependencies, leftover starter images) from scaffolding; a couple of unused settings and orphaned database reads.
- **Test blind spot:** most audit-log tests can pass on old data from previous runs — tighten them so a broken audit write actually fails a test.
- **Documentation drift:** several design docs describe an older/incorrect version of how permissions and settings work (e.g. claim permissions read the login token — they don't; they read the live database, which is safer). All three stress: fix the docs so future reviewers aren't misled.
- **Speed (no rush):** a few queries fetch whole tables to show five rows, and login details are re-checked 2–3x per page. Easy wins once the company grows; fine at today's size. Confirm with real measurements before adding database indexes.

6 What's already solid — don't "fix" these

All three audits explicitly confirmed the following are working as intended, not bugs:

- **Managers seeing their team's salary summary** is the intended, tested feature — an old doc line that says otherwise is the thing that's out of date (bank/tax/ID details stay hidden).
- **Permissions can't be faked with a stale login token** — the system always reads the live database role (all three checked this independently).
- **File downloads, cross-employee access, and salary self-edits** were all tested and confirmed properly locked down.
- **No made-up framework code, clean typing, safe database migrations, and a genuine security test suite** that checks real properties against a real database.

Confidence tags: **All 3** found by all three systems (highest confidence) · **2 of 3** two systems · **Codex** **Fable** **Sol** surfaced by one system (the value of running three — each caught things the others didn't).

KushHR internal audit summary · Code status: **GO with residual external watch** — no reason to pull the app; fix the "Must fix" list before signing off the audit gate. Production-readiness: **2.5/5 pilot-grade** — land the Section-4 items before real staff data. · Three independent AI reviews (Codex GPT-5 · Fable 5/Opus · Sol GPT-5.6). Generated from *docs/checks/{codex,fable,codex-sol}-audit-*.md*. Engineering action list with file references: *docs/checks/audit-remediation-plan.md*.